

SIH26231 — App Design System & Build Prompts

For handoff to Antigravity / DeepSeek / any AI code-builder

Stack assumed (from the implementation plan): **Flutter (Dart)**, on-device OpenCV, Google ML Kit, Firebase, SQLCipher. If your builder insists on a different stack, the *design system and prompt logic* below still applies — only the package names in Section 6 change.

1. Roles: does Admin belong in the mobile app? — No, and here's the reasoning

You're right to question this. Walking through it properly:

Officer — field-only role, needs the app constantly, needs it fast and minimal. **Supervisor** — mostly field-adjacent, occasionally needs to approve something *while mobile* (a bulk export request, a high-stakes co-sign) — so a **thin slice** of supervisor function belongs in the app, not the full admin toolkit. **Auditor** — a desk-review role by nature. Reviewing chain integrity, searching historical records, reading the audit-of-the-audit-log — this is spreadsheet-and-dashboard work, not something anyone does standing in a field. **Web-only. Admin** — device provisioning, officer roster management, reference-card issuance, threshold recalibration. This is back-office administration that happens at a desk, on a real keyboard, with a real screen — and critically, **it should never be reachable from a device that could be lost or stolen in the field**. Putting admin capability on the mobile client only increases the attack surface of the thing most likely to go missing. **Web-only, deliberately.**

Decision: build two things, not one —

1. **Officer/Supervisor mobile app** (Flutter) — this document
2. **Admin/Auditor web console** (can be a simple React or even server-rendered dashboard, doesn't need to be fancy — Firebase Auth + Firestore is enough for a hackathon-grade version)

This isn't a scope-cut — it's a correct security boundary, and it's a good line to say out loud in your pitch: *"We deliberately kept administrative capability off the field device."*

2. OOP Domain Model

Don't build deep inheritance trees (Officer extends Supervisor extends User extends Person) — this gets brittle fast). Use **one User class with a Role enum**, and **composition** for capability, not inheritance. This is the practical OOP call for a codebase two people will maintain under deadline pressure.

```

enum Role { officer, supervisor, auditor, admin }

class User {
    String userId;
    String badgeNumber;
    String department;
    Role role;
    String deviceId;          // bound device
    DateTime provisionedAt;
}

class Device {
    String deviceId;
    String hardwareKeyFingerprint; // from Keystore/Secure Enclave
    String? assignedUserId;
    DeviceStatus status;          // active | revoked | lost
}

class ReferenceCard {
    String serial;
    String qrPayload;
    String? issuedToOfficerId;
    CardStatus status;            // active | retired | reported_lost
}

class TestSession {              // mutable, in-progress, lives only on-
device until finalized
    String sessionId;
    KitType kitType;              // NDDK | PCDK | KDK
    DateTime reactionStartedAt;
    List<CapturedFrame> burstFrames;
    CalibrationResult? calibration;
}

class CalibrationResult {
    double deltaE;
    double confidence;
    bool cardDetected;
    bool withinReactionWindow;
    bool qualityPassed;
}

class ClassificationResult {
    ResultCategory category;      // positive | negative | inconclusive
}

```

```

    double confidence;
    List<String> interferentWarnings; // e.g. "Levamisole may cause false
positive on this reagent"
}

class TestRecord {                                // immutable once signed – this is the
evidence object
    String testId;
    String prevHash;
    String recordHash;
    User officer;
    Device device;
    ReferenceCard card;
    TestSession session;
    ClassificationResult result;
    GeoPoint? location;
    bool locationConfirmed;
    DateTime timestamp;
    Signature deviceSignature;
    Signature officerSignature;
    SyncStatus syncStatus;
}

class Signature {
    String signerId;
    String algorithm;    // ECDSA-P256
    String signatureBytes;
    DateTime signedAt;
}

class AuditLogEntry {                            // itself hash-chained, see implementation
plan §4.6
    String logId;
    String prevLogHash;
    String actorId;
    AuditAction action;    // view | export | search
    String? targetTestId;
    DateTime timestamp;
}

```

Key methods to design around (not exhaustive, but these are the ones that matter):

- `TestSession.runCalibration()` → returns `CalibrationResult`, never mutates the raw captured frames

- `TestRecord.sign(deviceKey, officerKey)` → only callable once; record becomes immutable after
- `TestRecord.appendToChain(previous)` → computes `recordHash`, enforces the hash-chain invariant
- `HashChain.verify()` → walks the whole local chain, returns first broken link if any (this is your demo-able integrity check)

3. Design System — colour, type, spacing (GIGW 3.0 / WCAG 2.1 AA compliant)

GIGW 3.0 (India's official government app standard) mandates **WCAG 2.1 Level AA**, which means **4.5:1 minimum contrast** for normal text and UI text. Every pairing below is computed, not guessed.

3.1 Colour palette

Token	Hex	Use	Contrast (computed)
<code>bg.base</code>	 <code>#F7F9FC</code>	App background (light, cool, not stark white)	Text on it: 15.65:1
<code>bg.surface</code>	 <code>#FFFFFF</code>	Cards, sheets	—
<code>text.primary</code>	 <code>#1A1F29</code>	Body text, headings	15.65:1 on <code>bg.base</code>
<code>text.secondary</code>	 <code>#5B6472</code>	Supporting/meta text	~4.6:1 on <code>bg.base</code> — use only for text ≥14sp, verify with a contrast checker before shipping
<code>border.default</code>	 <code>#E1E5EB</code>	Dividers, input borders	non-text, no AA requirement
<code>primary</code>	 <code>#1E4B8F</code>	Primary buttons, active nav, links	White text on it: 8.55:1
<code>primary.pressed</code>	 <code>#163765</code>	Pressed/active state of primary	darker, safe
<code>secondary</code>	 <code>#445266</code>	Secondary buttons	—

Status colours — deliberately NOT traffic-light-naïve. "Positive" here means *drug detected*, which is an alert condition, not a "good" green outcome — so:

Result	Background tint	Text/icon colour	Contrast (computed)
Positive (drug detected — alert)	 #FDECEA	 #B3261E	5.71:1
Negative (none detected — clear)	 #E8F5E9	 #1B6E2F	5.62:1
Inconclusive (needs retest/lab)	 #FFF4E0	 #9E5B00	4.88:1

Rule, not optional: never convey result by colour alone. Every status badge pairs colour + icon (✓ / ✕ / !) + text label, so the app is usable for colourblind officers too — worth mentioning explicitly in your deck as a GIGW/WCAG compliance point.

3.2 Typography scale

Modular scale, ratio 1.25 (a "major third" — standard, readable, not decorative):

Token	Size	Use
<code>display</code>	28sp	Result screen headline (POSITIVE/NEGATIVE/INCONCLUSIVE)
<code>title</code>	22sp	Screen titles
<code>subtitle</code>	18sp	Section headers
<code>body</code>	16sp	Body text, form labels
<code>caption</code>	13sp	Metadata, timestamps, hash fragments

3.3 Spacing — 8dp grid

Base unit = 8dp. All spacing is a multiple of it: **4, 8, 16, 24, 32, 40, 48**. Never use an arbitrary value like 13dp or 22dp — this single rule is what keeps every screen visually consistent without a designer manually checking each one.

- Screen side margins: **16dp**
- Gap between stacked elements: **16dp** default, **8dp** for tightly related items (label+input)
- Section spacing: **32dp**

3.4 Touch targets — the "nothing overlaps" math

- Minimum touch target: **48×48dp** (below this, mis-taps happen — this matters even more for officers wearing gloves or working one-handed)
- **Capture shutter button** (highest-stakes, most-tapped control): **72dp diameter**, circular, centered horizontally, **32dp** clearance above the bottom safe-area inset
- Any two adjacent interactive elements must have **≥8dp clearance** between their touch-target boundaries, not their visible boundaries — this is the rule that actually

prevents overlap and accidental double-hits

- Primary CTA buttons: full width minus margins ($(\text{screen_width} - 32\text{dp})$), height **56dp**
- Icon-only buttons (back, close): **48×48dp** hit area even if the icon glyph itself is smaller

3.5 Layout grid

Single-column mobile layout. 16dp side margins, 8dp internal gutter for any side-by-side elements (e.g., two stat chips). No fixed multi-column grid needed at phone width — reserve that only if you build a tablet/supervisor layout later.

4. Screen Inventory (Officer + Supervisor mobile app)

#	Screen	Purpose	Role
1	Splash	Brief, checks device provisioning status	All
2	Login	Badge ID + PIN (+ optional biometric)	All
3	Home/Dashboard	Recent tests, "New Test" CTA, sync status	Officer, Supervisor
4	Kit Selection	NDDK / PCDK / KDK	Officer
5	Reference Card Scan	QR scan to validate card issuance	Officer
6	Reaction Timer	Countdown to valid reading window	Officer
7	Capture	Live camera + guide overlay, minimal chrome	Officer
8	Processing	Brief on-device calibration/classification	Officer
9	Result	Category + confidence + interferent flags + "lab confirmation required"	Officer
10	Record Confirm	Case number/notes, triggers dual-sign + chain append	Officer
11	Test History	Own records, search/filter	Officer
12	Record Detail	Full record view: image, metadata, hash, chain, sync state	Officer, Supervisor
13	Sync Status	Pending queue, manual retry	All
14	Approval Queue	Co-sign / bulk export approval	Supervisor only
15	Profile/Settings	Officer info, device info, logout	All

5. Build Prompts — copy-paste these directly

Read this before you paste anything: the reason your last builder "built simulations" is almost certainly that the prompts didn't forbid it. Every prompt below explicitly bans mocked/random/hardcoded results — leave that instruction in, even if it feels redundant.

5.1 Prompt — Project setup

```
Create a new Flutter project named "field_test_companion" targeting Android
and iOS.
Use Riverpod for state management (not Provider, not Bloc – Riverpod's
compile-time
safety catches null/state bugs earlier, which matters for a small team).
Add these dependencies to pubspec.yaml:
  camera, geolocator, google_mlkit_barcode_scanning, sqflite,
sqlcipher_flutter_libs,
  flutter_secure_storage, crypto, pointycastle, flutter_local_notifications,
  telephony (or flutter_sms), cloud_firestore, firebase_auth,
firebase_storage,
  firebase_core

Set up sound null safety strictly. Do NOT use the `!` null-assertion operator
anywhere in camera, GPS, or file I/O code paths – these must handle
null/failure
explicitly with a fallback UI state, never crash.

Wrap app startup in runZonedGuarded() and set FlutterError.onError to log
uncaught
exceptions to a local file instead of letting the app crash silently – this
is
required, not optional.
```

5.2 Prompt — Capture screen (camera + GPS)

Build a Flutter screen called CaptureScreen with a full-screen camera preview using the `camera` package. Minimal chrome only: a thin top bar with a back button (48x48dp tap target), and nothing else except what's below – no extra text, no decorative elements, this screen exists for one purpose.

Requirements:

1. Initialize the camera controller in initState(), and MUST dispose it in dispose().
Wrap initialization in try/catch – if camera permission is denied or camera fails to initialize, show a full-screen fallback message with a "Grant Camera Permission" button, never leave a blank/crashed screen.
2. On top of the live preview, draw an overlay guide (a semi-transparent frame) showing where the reference card should sit. Do this with a CustomPainter, not a static image asset, so it can show live pass/fail state (see below).
3. Every ~200ms, run these checks on the current camera frame:
 - Reference card detection: use OpenCV ArUco marker detection (via a platform channel to native OpenCV, or the opencv_dart package if you've verified it's stable in your Flutter version) to find all 4 corner markers of the printed reference card. Do NOT fake this with a fixed-position rectangle – it must actually detect the markers in the live frame.
 - Blur check: compute Laplacian variance on the frame (OpenCV). Below a threshold (tune this empirically, start around 100) = "too blurry."
 - Exposure check: flag if the histogram is clipped at either extreme.Update the overlay in real time: green outline + checkmark when all pass, red outline + specific text reason when any fail (e.g. "Card not fully in frame" / "Image too blurry" / "Too dark").
4. The capture button (72dp diameter, centered, 32dp above bottom safe area) is DISABLED (visually greyed, not tappable) until all three checks pass. This is

a hard gate, not a suggestion.

5. On tap, capture a BURST of 4 frames over ~400ms (not a single photo).
Store all

4 in memory temporarily, downsize/dispose the rejected ones immediately
after
selecting the best frame (least specular highlight over the reaction zone)
to
avoid memory pressure – holding 4 full-res frames indefinitely is a common
cause of the app freezing/crashing on lower-end devices.

6. Simultaneously, fetch GPS via the `geolocator` package with a 5-second
timeout.

If it fails or times out, do NOT block capture – proceed and mark the
record

`locationConfirmed = false`. Never show a crash or infinite spinner for
GPS

failure; fail open on capture, fail closed on trust (flag it, don't block
it).

Do not simulate detection results. Do not return a hardcoded or random
pass/fail –
every check must run against real pixel data from the live frame.

5.3 Prompt — Reference card / QR scan screen

Build a Flutter screen called CardScanScreen that scans the QR code printed on the reference card BEFORE the capture flow starts, to validate the card is genuine and currently issued.

Use `google_mlkit_barcode_scanning` (on-device, works fully offline, no API key).

Requirements:

1. Live camera preview with a bounding-box overlay showing detected QR codes.
2. Debounce: only accept a scan after the SAME payload string has been read on 3 consecutive frames. A single-frame read is not enough – this prevents misreads from motion blur being accepted as valid.
3. Validate the payload format before accepting: expected format is a fixed prefix like "NCBCARD-" followed by a serial number and checksum digit. Reject anything that doesn't match with a clear "Invalid or unreadable card" message and a retry button – never crash on unexpected QR content.
4. If no valid QR is detected within 3 seconds, show a "Try toggling flashlight" prompt with a torch-toggle button (camera package supports this).
5. On valid scan, call the backend (or local cache if offline) to check the card serial against the issuance log. If offline, allow proceeding but flag the record `cardVerificationPending = true` for later server-side reconciliation.
6. On success, navigate to the Reaction Timer screen, passing the validated card serial forward.

5.4 Prompt — Calibration & classification engine

Build a pure-Dart (no UI) service class called CalibrationEngine with a method:

```
Future<ClassificationResult> classify(CapturedFrame frame, KitType kit)
```

This MUST run on a separate isolate (use Dart's `compute()` function), not the main UI isolate — ΔE calculation and color-space conversion are CPU-heavy and will freeze the UI thread if run inline, which looks like an app crash/hang to the user even though it isn't one.

Steps the function must actually perform (no shortcuts, no hardcoded output):

1. Locate the 4 ArUco corner markers on the captured frame (reuse detection from capture screen, or re-run if needed).
2. From the known positions of the grayscale reference patches on the card, compute a white-balance/illumination correction matrix.
3. Apply that correction to the pixels in the reaction-zone region of interest.
4. Convert the corrected reaction-zone average color from sRGB to CIE Lab.
5. Compute ΔE_{2000} distance between the corrected color and the reagent's documented reference color for this KitType (store these reference values in a versioned config file, not hardcoded inline, so they can be recalibrated later without a code change).
6. Apply the threshold (from the same versioned config) to output: `ResultCategory.positive` / `.negative` / `.inconclusive`
7. Confidence score = a function of (ΔE margin from threshold) AND (image quality score from capture checks) — both must contribute, not just one.
8. Look up the KitType + detected category against a known-interferents table (populate from the false-positive-rate research — e.g. levamisole/lidocaine for certain reagents) and attach any matching warnings to the result.

Write a unit test that feeds this function two visibly different colored test images and asserts it returns two DIFFERENT ΔE values — this is your proof to yourself that it's doing real computation, not returning a fixed number.

5.5 Prompt — Result screen & alert handling

Build a Flutter screen called ResultScreen shown immediately after classification completes.

Layout:

- Large status badge at top using the exact colour tokens from the design system

(Positive: bg #FDECEA / text #B3261E, Negative: bg #E8F5E9 / text #1B6E2F, Inconclusive: bg #FFF4E0 / text #9E5B00) – pair every badge with an icon

AND

text label, never colour alone.

- Confidence percentage shown as plain text, not just a colour intensity.
- If any interferent warnings were returned, show them in a clearly separated callout box below the badge – do not bury this in fine print.
- A persistent, non-dismissible banner (cannot be swiped away, always visible on

this screen): "Presumptive result only – laboratory confirmation required."

This must be structurally present in the widget tree, not a toast that disappears after a few seconds.

- Two buttons: "Retake" (returns to Capture screen, discards this attempt) and

"Confirm & Record" (proceeds to Record Confirm screen, only enabled – do not

let the officer silently skip past the presumptive-only banner).

Alert handling after the scan completes:

- If the app is in the foreground, show the ResultScreen directly – no system notification needed.

- If classification finishes while the app was backgrounded (e.g. during a sync

operation), fire a LOCAL notification via flutter_local_notifications (not a

remote push – this is on-device work, doesn't need network) saying

"Test result ready – tap to review," deep-linking back into ResultScreen.

- If classification fails entirely (e.g. calibration engine throws), do NOT show

a raw error or crash – show a specific "Could not process image, please

retake" screen with a Retake button, and log the actual exception locally.

5.6 Prompt — Login / sign-in screen

Build a Flutter screen called LoginScreen. This is a GOVERNMENT PROVISIONED-DEVICE

app, not a consumer app – there is no self-service "Sign Up." Fields:

- Badge Number (pre-registered by Admin via the web console – this field identifies the officer, it does not create a new account)
- PIN (numeric, minimum 6 digits; on first login, prompt to SET a PIN; hash with PBKDF2 or Argon2 before ever storing or transmitting – never store or send a raw PIN)
- Optional: biometric unlock (local_auth package) as a convenience layer ON TOP of the PIN, not a replacement for it – biometric only unlocks the locally stored PIN-derived key, it never becomes the signing credential itself.

On submit:

1. Check the device's hardware key fingerprint (from Keystore/Secure Enclave) against this badge number's provisioning record (cached locally after first successful online login, for offline login support).
2. If the device isn't provisioned to this officer, show a clear "This device is not registered to your badge number – contact your administrator" message. Do not allow login to proceed.
3. On success, derive the officer signing key from the PIN (PBKDF2, with a per-officer salt stored via flutter_secure_storage) and hold it in memory only for the session – never persist the derived key to disk.

5.7 Prompt — Crash-prevention checklist (apply to the whole app, not one screen)

Apply these rules across the entire codebase, not just individual screens:

1. Every camera/GPS/file-I/O call is wrapped in try/catch with a specific, user-visible fallback state – no bare `catch (e) {}` blocks that swallow errors silently, and no unhandled exceptions that propagate to a crash.
2. Every StatefulWidget that opens a resource (camera controller, stream subscription, animation controller) disposes it in dispose(). Missing disposal is the most common cause of memory-leak-driven crashes in camera-heavy Flutter apps.
3. Never run ΔE calibration, ArUco detection, or any OpenCV operation on the main isolate – always via compute() or a dedicated isolate.
4. Set FlutterError.onError and runZonedGuarded at app startup to catch and log uncaught errors instead of hard-crashing.
5. Test explicitly for: camera permission denied, location permission denied, airplane mode/no connectivity during sync, low storage during capture, rapid repeated taps on the capture button (debounce it), and backgrounding the app mid-capture. Each of these must degrade gracefully, not crash.

6. Package reference (what each one is actually for)

Package	Purpose	Cost
<code>camera</code>	Live preview, frame capture	Free
<code>google_mlkit_barcode_scanning</code>	QR/barcode reading, on-device	Free
OpenCV (native/ <code>opencv_dart</code>)	ArUco detection, blur check, colour-space conversion	Free
<code>geolocator</code>	GPS	Free
<code>sqflite</code> + <code>sqlcipher_flutter_libs</code>	Encrypted local DB (the hash-chain ledger)	Free
<code>flutter_secure_storage</code>	Wraps Android Keystore / iOS Keychain for key material	Free
<code>crypto</code> (Dart)	SHA-256	Free
<code>pointycastle</code>	PBKDF2/Argon2 key derivation, ECDSA	Free
<code>telephony</code> / <code>flutter_sms</code>	Native SMS anchor, no gateway	Free (normal SIM SMS cost)
<code>flutter_local_notifications</code>	On-device result alerts	Free
<code>firebase_*</code>	Backend sync, auth, storage	Free (Spark plan)
Riverpod	State management	Free

7. Testing checklist

- **Unit tests:** ΔE calculation (feed known colour pairs, assert expected distance), hash-chain integrity (assert a tampered record breaks verification), PIN derivation (deterministic for same input).
- **Widget tests:** Capture screen gates the shutter button correctly under each quality-check failure state; Result screen always renders the presumptive-only banner.
- **Manual field matrix:** test capture under — direct sunlight, indoor fluorescent, low light + flash, across at least 2–3 different phone models your team can access.
- **Failure-mode tests:** camera permission denied, GPS denied, airplane mode mid-sync, rapid double-tap on shutter, backgrounding mid-classification, low storage.

This complements the earlier implementation plan (tech stack/backend/security) and the council reviews. Hand Sections 5.1-5.7 to your code-builder one at a time, in order — each depends on the previous screen existing.